15.2.2 Read-through

Instead of managing both the database and the cache, the application layer interacts only with the cache system; the database management logic being hidden behind the caching API. Compared to the cache-aside use case, the data access logic is simplified since there is only one data source to communicate with.

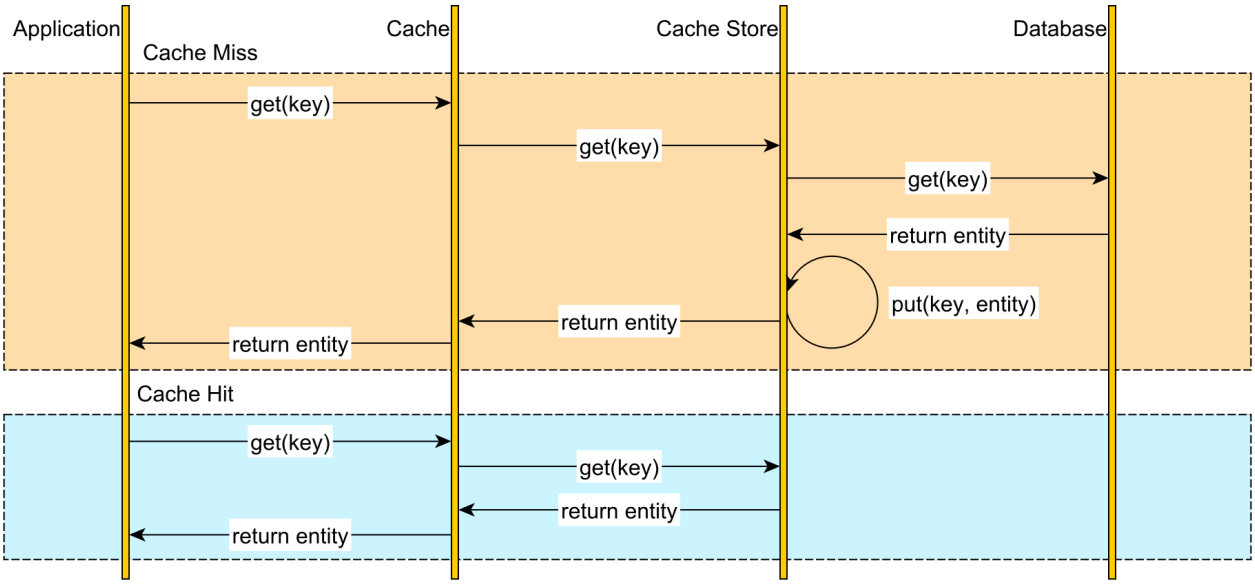

Figure 15.3: Read-through synchronization strategy

When fetching an entity, the cache checks if the requested entity is already contained in the cache store, and, upon a cache miss, the entity is loaded from the database.

15.2.3 Write-invalidate

If the entity is modified, the cache propagates the change to the underlying database and removes the associated entry from the cache. The next time this entity is requested, the cache system is going to load the latest version from the database.

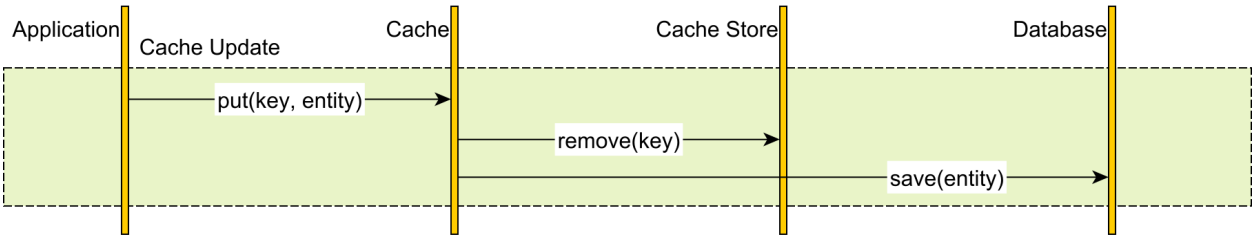

Figure 15.4: Write-invalidate synchronization strategy

15.2.4 Write-through

If the entity is modified, the changed is propagated to the underlying database and the cache as well.

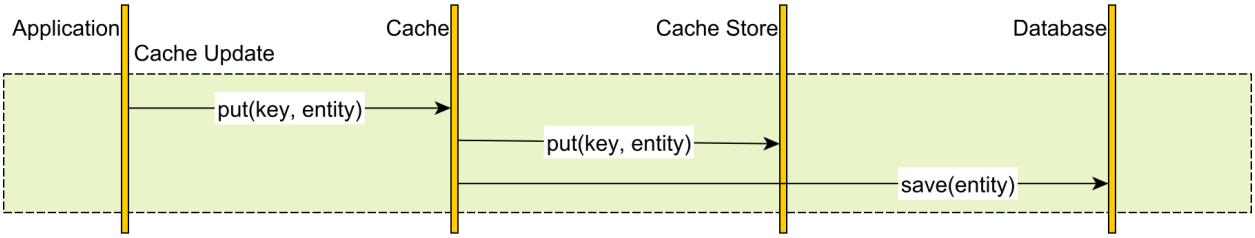

Figure 15.5: Write-through synchronization strategy

If the caching layer supports JTA transactions, the cache and the database can be committed at once. Although XA transactions can simplify development, the two-phase commit protocol incurs a significant performance overhead.

An alternative is to use soft locks on the cache side to hide the cache entry modification until the database transaction is committed, so that, until the lock is released, other concurrent transactions must load the entity from the database.

15.2.5 Write-behind

If strong consistency is not mandated, the change requests can be enqueued and flushed at once to the database.

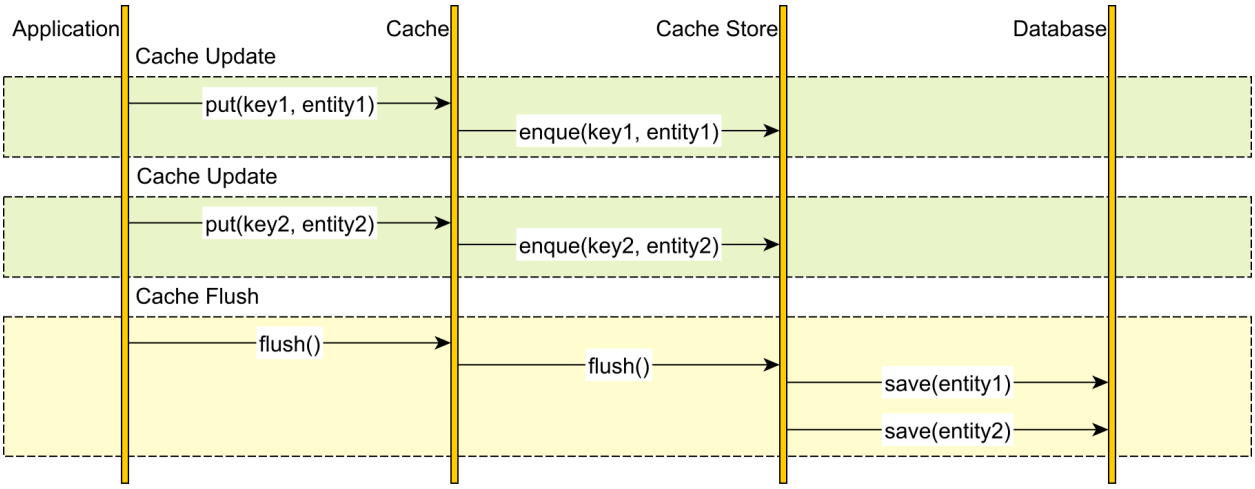

Figure 15.6: Write-behind synchronization strategy

This strategy is employed by the JPA Persistence Context, all entity state transitions being flushed towards the end of the currently running transaction or prior to executing a query.

15.3 Database caching

As explained at the beginning of this chapter, most database engines make use of internal caching mechanisms to speed up read and write operations. The most common database cache component is the in-memory buffers, but there might be other components as well such as the execution plan cache or query result buffer. Even without a database cache, the underlying operating system may offer caching for data pages.

Unlike application-level caches, the database cache does not compromise data consistency. Being both read-through and write-through, the database cache is transparent to the data access layer. Even with the advent of SSD (solid-state drive), disks still have a much higher latency than RAM. For this purpose, it makes much sense to load frequently-accessed data from memory, rather than going to disk.

Oracle

Oracle has multiple mechanisms for caching, such as:

- Buffer pool - storing blocks of data that are loaded from the underlying disk drive.
- Shared pool - storing parsed SQL statements, schema object metadata, sequence numbers.
- Large pool - stores results for parallel queries, large I/O buffers that are used for recovery management and backup or restore procedures.
- Result cache - stores results for SQL queries (when using the `RESULT_CACHE` query hint) and PL/SQL functions (when using the `RESULT_CACHE` directive).

On Unix systems, all I/O goes through the OS page cache. However, the same data is going to be cached in the Buffer pool, therefore data blocks are cached twice. For this reason, direct I/O[a] is desirable because it can bypass the file system cache, and the OS page cache can be used for other system processes.

There are also use cases when Oracle does not use the Buffer pool for caching data blocks (e.g. `TEMP` tablespace operations, `LOB` columns using the `NOCACHE` storing option), in which case the operating system cache may be suitable for speeding up read and write operations.

Although each caching structure can be configured manually, it is often a good idea to leave this responsibility to the automatic memory management[b] mechanism, which is enabled by default.

[a] http://docs.oracle.com/database/121/TGDBA/pfgrf_os.htm#TGDBA94410
[b] https://docs.oracle.com/database/121/TGDBA/memory.htm#TGDBA505

SQL Server

To provide very low transaction response times, SQL Server strives for reducing I/O operations (which are a source of performance-related issues in many database systems). For this reason, the database engine tries to use as much system memory as possible so that frequently-accessed data and index disk pages are served from RAM rather than the disk drive.

Upon startup, SQL Server allocates a portion of the system memory and uses it as a buffer pool. The buffer pool is divided into multiple pages of 8KB. Both data and index pages are read from disk into buffer pages, and, when the in-memory pages are modified, they are written back to disk.

SQL Server 2014 supports buffer pool extensions[a], which allow it to use SSD drives to increase the buffer cache size beyond the capabilities of the current system available memory.

[a] https://msdn.microsoft.com/en-us/library/dn133176.aspx

PostgreSQL

For improving read and write operation performance, PostgreSQL relies heavily on the underlying operating system caching capabilities. However, most operating systems use a LRU (least recently used) page replacement policy which is unaware of the data access patterns or other database-related considerations.

For this reason, PostgreSQL defines a shared buffers structure which stores disk pages into 8KB in-memory page cache entries. The shared buffer size is controlled via the `shared_buffers` configuration property. Unlike the OS cache, the shared buffers use a LFU (least frequently used) algorithm called clock sweep which counts the number of times a disk page is used. The more often a disk page is being used, the longer it is going to linger in the shared buffer database internal cache.

That being said, the shared buffer structure is more useful for storing frequently-accessed data blocks, while the operating system cache can be used for everything else. The shared buffer cache should not be set too high because the database engine requires memory for other operations as well (sorting, hashing, building indexes, vacuuming).

Although the shared buffers structure is very important for speeding up reads and writes, it is good practice to limit the shared buffer size[a] to the size of the current working set, therefore leaving enough memory for other database-related tasks.

[a] http://www.postgresql.org/docs/current/static/runtime-config-resource.html

MySQL

MySQL uses its internal buffer pool to cache data and indexes. The buffer pool is implemented as a linked list of memory pages. If the buffer pool size is smaller than the overall InnoDB tablespace size, a LRU-based algorithm is going to be used to deallocate older page entries.

The pool size is given by the `innodb_buffer_pool_size` configuration property which, ideally, should be adjusted so that it can hold all data and indexes in memory. Care must be taken to allow enough memory for the OS, as well for other MySQL structures and processes (e.g. threads allocated for each individual connection, sort buffers, query cache).

On Linux, to avoid double buffering caused by the operating system caching mechanism, the `innodb_flush_method`[a] configuration property should be set to `O_DIRECT`.

Nevertheless, the OS cache is useful for storing the InnoDB transaction log (used for ensuring ACID transactions), the binary log (used for database replication), and other MySQL structures that are not covered by the InnoDB buffer pool.

[a] http://dev.mysql.com/doc/refman/5.7/en/innodb-parameters.html#sysvar_innodb_flush_method

Essential, but not sufficient

Database caching is very important for a high-performance enterprise application. However, database caching only applies to a single node, and, if the database size is bigger than the capacity of a single node, then this solution alone is no longer sufficient. One workaround is to use database sharding, but that is not without challenges.

Even if database caching improves performance considerably, the networking overhead still plays a significant role in the overall transaction response time. If the application operates on graphs of entities, fetching an entire graph might require lots of joins or many secondary select statements. For this reason, it makes sense to cache the whole entity aggregate and have it closer to the application layer.

If the enterprise system relies only on the database system alone to serve read requests, the database becomes a single point of failure. This availability can be increased by using database replication. However, if all database nodes are collocated (to reduce the synchronization overhead caused by networking latency), the database system can still become unavailable if the data center is facing a sudden power outage.

For all these reasons, it is good practice to use an application-layer caching solution to address database caching limitations.